

SoftyFy

Project & Technical Report — Streaming PWA hosted on Google Drive · 9 Sep 2026

React + TypeScript + Vite

Progressive Web App

Google Drive backend

Custom Service Worker

Render deploy

37 songs · ~349.5 MB

37

songs streamable

3

libraries

349.5 MB

media total

0

network on replays

1. What is SoftyFy?

SoftyFy is a **Progressive Web App** that streams songs. It is a **static frontend** (React + Vite) deployed at <https://softyfy.onrender.com>. The GitHub repository ([pradipME/SoftyFy](#)) contains **code only** — all audio and cover artwork live on **Google Drive**. A custom **Service Worker** makes repeat playback instant and offline-capable.

2. Architecture

Layer	Technology	Role
Frontend	React + TypeScript + Vite	UI, player, library views
PWA engine	vite-plugin-pwa (injectManifest)	Generates <code>sw.js</code> from hand-written <code>src/sw.ts</code>
Audio storage	Google Drive (official API)	Stream + cache source for all 37 songs
Artwork	Google Drive thumbnail endpoint	Cover images per album
Upload tooling	rclone	Uploads media to Drive from local machine
Hosting	Render (static, free tier)	Serves the app shell; auto-deploys from <code>main</code>
Quality	Vitest (44 tests) + oxlint	Logic + component tests, linting

3. Content pipeline — how a song is added

- Source selection** — a YouTube link or a Spotify album (e.g. *Yours Truly*, 11 tracks) is chosen and its track list is retrieved (rclone `lsjson`).
- Upload** — `scripts/drive-upload.mjs` uploads the audio file and cover art into the Drive folder with rclone.
- Linking** — `scripts/link-drive-songs.mjs` reads Drive file IDs and writes the stream URL + cover URL into `frontend/src/data/songs.ts`.
- Deploy** — a push to `main` triggers an automatic Render redeploy; the Service Worker updates itself on the next visit.

4. The playback fix that made everything work

4.1 The bug

Google Drive's public sharing link (`drive.usercontent.google.com/download?id=...`) returned **HTTP 403 Forbidden** in every real browser while *working* in curl. Root cause: browsers attach `Sec-Fetch-Site: cross-site` to cross-origin media requests, and Drive's `usercontent` endpoint rejects any such request. Cover images (`thumbnail?id=...&sz=w1000`) were NOT affected, only audio.

Candidate URL	Result in browser
<code>drive.usercontent.google.com/download?id=...</code>	403 — cross-site blocked
<code>drive.google.com/uc?export=open ...</code>	303 → 403
<code>lh3.googleusercontent.com/d/<ID></code>	404
Service-worker proxy	CORS-blocked (no ACAO header)
<code>drive.google.com/thumbnail?id=...</code> (covers)	200 image/jpeg — works
<code>googleapis.com/drive/v3/files/<ID>?alt=media&key=...</code>	200/206 audio/mpeg — works, chosen

4.2 The chosen solution

Stream via the **official Drive API** using a public API key baked into the bundle:

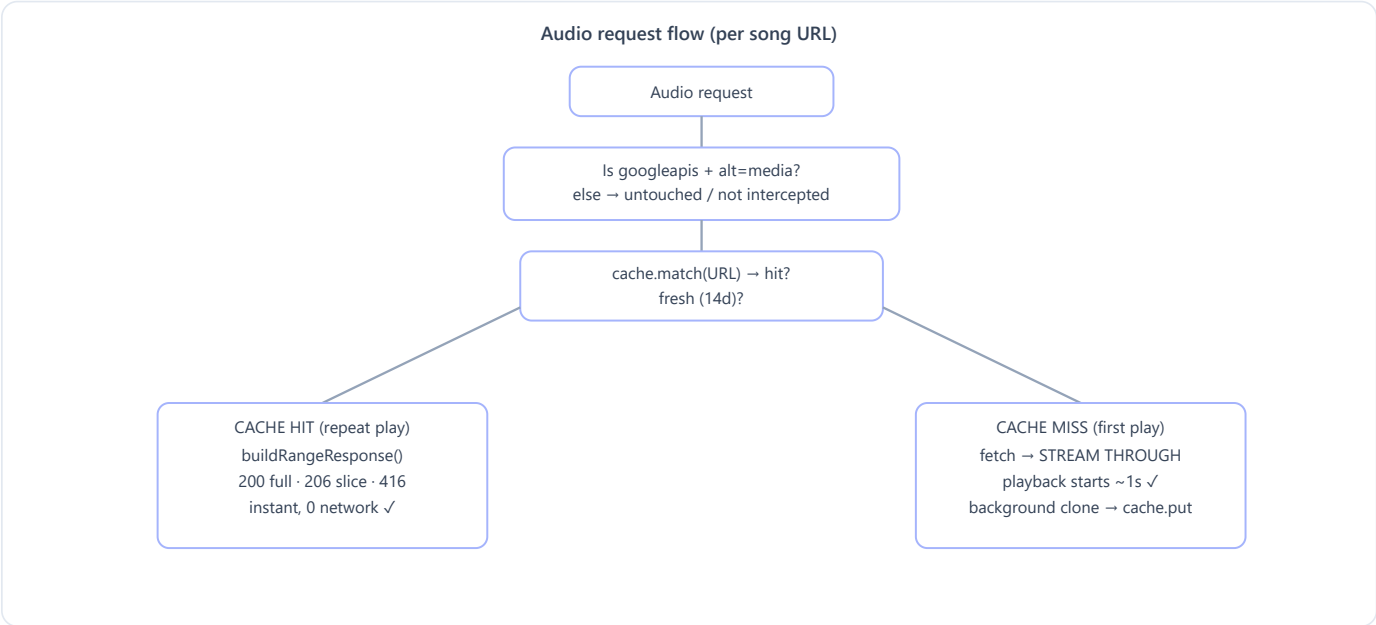
```
https://www.googleapis.com/drive/v3/files/<FILE_ID>?alt=media&key=AIzaSyAxkYvJVfY_5HXwvej0lMi0yno613rtK8
```

- Honours `Range` requests (206) and sends permissive CORS headers (ACAO happily echoes `softyfy.onrender.com`).
- API keys do **not expire**; the key is *public by design* since it ships to every client. Real risks: key deleted/regenerated, or a future Google policy change on the official API.

5. Service Worker — caching design

A hand-written SW (`frontend/src/sw.ts`) is compiled by vite-plugin-pwa into `sw.js` . Google does not send cache directives for these streams (`cache-control: private, max-age=0` , no ETag), so without the SW **every play re-downloads the full ~8 MB file** — ~350 MB across the library. The SW fixes this while keeping first play fast.

5.1 Audio handler — stream first, cache in background



- **Cache miss** — the Google response is passed through **unchanged** (streaming), so playback starts immediately. Meanwhile the full body is copied in the background using `response.clone()` (taken *before* handover — cloning afterwards throws and used to silently break caching), stored with an `x-softify-cached-at` tag.
- **Cache hit** — the single cached file answers any request with correct byte-range semantics (**200** full file, **206** exact slice for `Range` , **416** for out-of-range) so **seeking mid-song works perfectly** even before downloading the track.
- **Hybrid** — seek requests which carry a real `Range` (mid-stream) pass straight through Google's native range handling.
- **Bounds** — max ~8 songs, 14-day freshness, oldest entry evicted. Quota errors are caught silently and playback continues from the network (safe fallback).

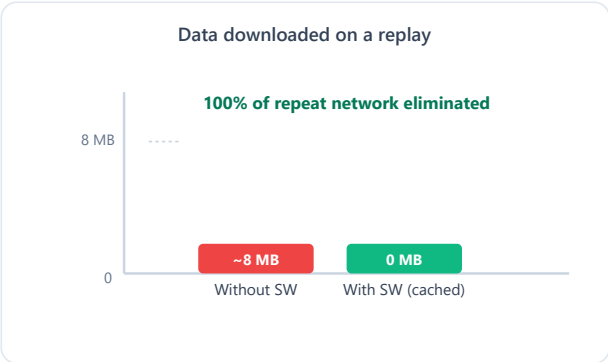
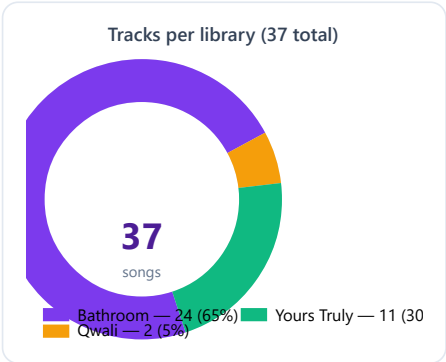
5.2 Covers handler

Cached on first sight in `softify-covers` ; `<img>` loads are no-cors/opaque and are stored + served unchanged. (The first workbox `CacheFirst` route silently never stored them — replaced by the same manual-handler pattern as audio.)

5.3 App shell & updates

Shell assets precached via `precacheAndRoute(self.__WB_MANIFEST)` ; `skipWaiting()` on install + `clients.claim()` on activate + `SKIP_WAITING` message → new versions activate instantly, `cleanupOutdatedCaches()` removes stale caches.

6. Charts



Bandwidth saved per session (37 songs, full listen through)			
Scenario	Downloads first play	Downloads each replay	Total for 3 full listens
Without Service Worker	~350 MB	~350 MB	~1,050 MB
With Service Worker	~350 MB	0 MB (cached)	~350 MB (-67%)

7. Verification

7.1 Test matrix (Headless Edge, local + LIVE)

Check	Local (dist)	Live (onrender)
First play (uncached)	streams, starts ~1 s; no cache entry during fetch	same; BG fill ~6–12 s in parallel
Second play	0 network — served [SW] from cache	0 network — cachedAt header present
Seek	to 90 s landed exactly (206 slice)	to 111 s landed (206 slice)
All traced responses	[SW] only — zero real outbound on replays	
Covers cache	entries populate after first load	

7.2 Notes on a bug found during verification

The first caching attempt eagerly `arrayBuffer()`-ed the whole file **before** responding — first play of a new song stalled until ~8 MB downloaded. Fixed to stream-through + background copy; the seek test (which assumed a fully-buffered file) was also updated to reflect real streaming usage.

8. Build, test & deploy

```
cd frontend
npm run build      # type-checks + builds dist/ incl. sw.js (~19066 B)
npm test          # 44/44 tsc + Vitest tests pass
npm run lint      # oxlint - clean (2 pre-existing Fast-Refresh warnings)
git push origin main # Render auto-redeploys; live sw.js verified
```

9. Key files

File	Purpose
frontend/src/data/songs.ts	All 37 songs with Drive API stream URLs
frontend/src/sw.ts	Custom Service Worker (stream + cache + range slicing)
frontend/vite.config.ts	PWA injectManifest configuration
frontend/scripts/drive-url.mjs	Shared Drive URL builder + API key (env overridable)
frontend/scripts/drive-upload.mjs	rclone-based uploader for media
frontend/scripts/link-drive-songs.mjs	Writes file IDs into songs.ts
frontend/package.json	Build/test/lint scripts + workbox-precaching dep

10. Limitations & trade-offs

- First listen of a never-heard track downloads the full file in the background even if the user stops after seconds — the deliberate price of instant replays.
- Cache is capped (≈8 songs) so mobile storage stays bounded; quota exhaustion degrades gracefully to network streaming.
- The API key cannot be revoked per-client; security comes from the key being public + Drive sharing settings. The main long-term risk is Google policy change, not key expiry.
- rclone's shared client-id retirement (2026) affects only new uploads, never playback.